1 INTRODUCTION

Verification and testing of concurrent programs is difficult, since one must consider all the different ways in which processes/threads can interact. *Model checking* [Clarke et al. 1983; Queille and Sifakis 1982] addresses this problem by systematically exploring the state space of a given program and verifying that each reachable state satisfies a given property. Applying model checking to realistic programs is problematic, however, since it requires capturing and storing a large number of global states. *Stateless model checking* [Godefroid 1997] avoids this problem by exploring the state space of the program without explicitly storing global states. A special run-time scheduler drives the program execution, making decisions on scheduling whenever such decisions may affect the interaction between processes. Stateless model checking has been successfully implemented in tools, such as VeriSoft [Godefroid 2005], CHESS [Musuvathi et al. 2008], and Concuerror [Christakis et al. 2013], the tool we use in this article.

While stateless model checking is applicable to realistic programs, it suffers from combinatorial explosion, as the number of possible interleavings grows exponentially with the length of program execution. There are several approaches that limit the number of explored interleavings, such as depth bounding [Russell and Norvig 2009] and context bounding [Musuvathi and Qadeer 2007]. Among them, *partial order reduction* (POR) [Clarke et al. 1999; Godefroid 1996; Peled 1993; Valmari 1991] stands out, as it provides full coverage of all behaviours that can occur in *any* interleaving, even though it explores only a representative subset. POR is based on the observation that two interleavings can be regarded as equivalent if one can be obtained from the other by swapping adjacent, non-conflicting (independent) execution steps. In each such equivalence class (called a *Mazurkiewicz trace* [Mazurkiewicz 1987]), POR explores at least one interleaving. This is sufficient for checking most interesting safety properties, including race freedom, absence of global deadlocks, and absence of assertion violations [Clarke et al. 1999; Godefroid 1996; Valmari 1991].

Existing partial order reduction approaches are essentially based on two techniques, both of which reduce the set of process steps that are explored at each scheduling point:

- The *persistent set* technique, that explores only a provably sufficient subset of the enabled processes. This set is called a *persistent set* [Godefroid 1996]; variations are *stubborn sets* [Valmari 1991] and *ample sets* [Clarke et al. 1999].
- The *sleep set* technique [Godefroid 1996], that maintains information about the past exploration in a so-called *sleep set*, which contains processes whose exploration would be provably redundant.

These two techniques are independent and complementary, and can be combined to obtain increased reduction.

The construction of persistent sets is based on information about possible future conflicts between threads. Early approaches analyzed such conflicts statically, leading to over-approximations and therefore limiting the achievable reduction. *Dynamic Partial Order Reduction* (DPOR) [Flanagan and Godefroid 2005b] improves the precision by recording actually occurring conflicts during the exploration and using this information to construct persistent sets on-the-fly, "by need". DPOR guarantees the exploration of at least one interleaving in each Mazurkiewicz trace when the explored state space is acyclic and finite. This is the case in stateless model checking in which only executions of bounded length are analyzed [Flanagan and Godefroid 2005b; Godefroid 2005; Musuvathi et al. 2008].

Challenge. Since DPOR is excellently suited as a reduction technique, several variants, improvements, and adaptations for different computation models have appeared in the literature [Flanagan and Godefroid 2005b; Kastenberg and Rensink 2008; Lei and Carver 2006; Saarikivi et al. 2012; Sen